



Московский государственный университет имени М.В. Ломоносова

Факультет вычислительной математики и кибернетики

Кафедра суперкомпьютеров и квантовой информатики

Задание 2

Группа: 523

Повжик Юрий Максимович

Москва, 2025

Содержание

1. Постановка задачи	3
2. Описание алгоритма	4
3. Результаты	6

1. Постановка задачи

Рассматривается набор множеств $S=\{S_1, S_2, \dots, S_N\}$, для некоторых пар (S_i, S_j) которого заданы условия вида $size_i \ size_j \ should_intersect$, где

- $size_i$ – размер множества S_i
- $size_j$ – размер множества S_j
- $should_intersect$ – 1, если множества пересекаются и 0 иначе.

Требуется построить конкретное наполнение множеств $S_1, \dots, S_N$ целыми числами, такое что:

1. Размеры множеств и условия пересечения максимально соответствуют заданным
2. Используется минимально возможное число уникальных элементов (не является строгим ограничением, но желательное свойство)

Пример входных данных:

	ids	sizes
0	0_1	46_121_1
1	0_2	46_43_1
2	0_3	46_265_1
3	0_4	46_133_1
4	0_5	46_267_1

2. Описание алгоритма

1. Индексация множеств

На первом этапе каждому уникальному множеству из входных данных сопоставляется внутренний числовой индекс.

Это упрощает дальнейшую работу с массивами, графами и ускоряет обработку.

2. Определение целевых размеров множеств

Для каждого множества определяется его целевой размер — максимальное значение размера, встречающееся во всех условиях, где это множество участвует.

Этот шаг выполняет сразу две задачи:

- гарантирует, что итоговое множество сможет удовлетворить все условия по размеру;
- защищает алгоритм от возможных ошибок или несогласованностей во входных данных.

3. Построение графа обязательных пересечений

Далее строится неориентированный граф пересечений:

- вершины графа соответствуют множествам;
- между двумя вершинами добавляется ребро, если соответствующие множества по условию обязаны пересекаться.

Таким образом, граф отражает все строгие требования на наличие пересечений между множествами.

4. Поиск компонент связности

В построенном графе находятся компоненты связности.

Каждая компонента представляет собой группу множеств, которые прямо или косвенно обязаны иметь общие элементы.

Каждой компоненте присваивается уникальный идентификатор `cid`.

Ключевое свойство:

- любые два множества внутри одной компоненты должны иметь пересечение;
- множества из разных компонент не имеют требований на пересечение и могут быть полностью разделены.

5. Формирование общего пула элементов для компоненты

Для каждой компоненты создаётся общий пул элементов, размер которого равен максимальному целевому размеру среди всех множеств этой компоненты.

Этот пул используется как источник общих элементов, которые будут гарантированно присутствовать во всех множествах компоненты.

6. Наполнение множеств элементами

Для каждого множества внутри компоненты выполняется следующее:

1. Множеству выдаются первые элементы из общего пула — это гарантирует пересечение со всеми другими множествами компоненты.
2. Если целевой размер множества превышает размер общего пула, недостающие элементы добавляются как новые уникальные значения, не используемые нигде ранее.

7. Гарантированное отсутствие пересечений между компонентами

Поскольку каждая компонента использует собственный набор элементов, множества из разных компонент не имеют общих элементов.

Это автоматически гарантирует отсутствие пересечений между ними.

Пример:

Даны множества 0, 1 и 2 и данные для них:

0 0_1 3_4_1

1 1_2 4_2_1

2 0_2 3_2_0

Присваиваем множествам индексы: 0 — 0, 1 — 1, 2 — 2.

Определяем размеры: 3, 4, 2.

Строим граф и получаем единственную компоненту связности: 0 — 1 — 2.

Создаем для нее общий пул элементов по максимальному размеру множеств равному 4.

Присваиваем множествам элементы по возрастанию:

$0 = \{1, 2, 3\}$, $1 = \{1, 2, 3, 4\}$, $2 = \{1, 2\}$.

Недочет: множества 0 и 2 пересекаются.

3. Результаты

Описанный алгоритм, реализованный на языке Python, с использованием входных данных triplets_all.csv показывает следующие результаты:

Всего множеств: 878
Всего условий: 385003
Время выполнения алгоритма: 0.252 сек
Ассигасу: 91.9785%
Уникальных элементов: 1118
Суммарный размер множеств: 78925
Общее время: 22.043 сек

Благодаря сильному пересечению исходных множеств алгоритм показывает хороший результат, как по времени работы, так и по точности решения.